6

EPSO: Evolutionary Particle Swarms

V. Miranda, *Fellow IEEE*[1,2], Hrvoje Keko[1], and Alvaro Jaramillo[1]

[1]INESC Porto, Institute of Engineering in Systems and Computers of Porto
[2]FEUP, Faculty of Engineering of the University of Porto, Portugal
vmiranda@inescporto.pt

Abstract. This chapter presents EPSO (Evolutionary Particle Swarm Optimization), as an evolutionary meta-heuristic that implements a scheme of self-adaptive recombination, borrowing the movement rule from PSO (Particle Swarm Optimization). Besides the basic model, it discusses a Stochastic Star topology for the communication among particles and presents a variant called differential EPSO or dEPSO. The chapter presents results in a didactic Unit Commitment/Generator Scheduling Power System problem and results of a competition among algorithms in an intelligent agent platform for Energy Retail Market simulation where EPSO comes out as the winner algorithm.

6.1 Introduction

This chapter presents and discusses a method that has been called EPSO – Evolutionary Particle Swarm Optimization, which has been claiming interesting properties of convergence and, especially, of robustness in convergence, when compared with classical Evolutionary Algorithms (EA) and also with Particle Swarm Optimization algorithms (PSO).

From a conceptual point of view, EPSO has an interesting feature: it allows a double interpretation on how it works, because it may be seen from two perspectives – either as a variant of PSO or as a variant of EA. The following presentation hopefully will help in understanding how the method works and how should one manipulate its characteristics to obtain better convergence characteristics in specific problems.

The name EPSO, as far as the author is aware of, has been first coined in 2002, in [1]. Subsequently, a few papers more presented some interesting results and applications [2–5]. However, more recently, some other publications have started to use the acronym EPSO for other type of algorithms, such as "extended PSO" [6], "enhanced PSO" [7] or even "emotional PSO" [8]. It is unfortunate that this may lead to some confusion to readers and researchers. We wish to make clear from the start that by EPSO we refer to a class of

V. Miranda et al.: *EPSO: Evolutionary Particle Swarms*, Studies in Computational Intelligence (SCI) **66**, 139–167 (2007)
www.springerlink.com © Springer-Verlag Berlin Heidelberg 2007

algorithms of the family of self adaptive EA, where the classical operators for recombination are replaced by a rule similar to the particle movement of PSO. Alternatively, we refer to a class of algorithms of the family of PSO where the weights associated to the components of the movement rule are made to evolve in a self-adaptive mechanism.

6.2 Evolutionary Algorithms

Let's recall that a general Evolutionary Algorithm may be organized in the following steps:

Procedure EA
initialize a random population P of μ elements
REPEAT
reproduction (introduce stochastic perturbations in the new population) – generate λ offspring...
...by *recombination*
...by *mutation*
evaluation – calculate the fitness of the individuals
selection – of μ survivors for the next generation, based on the fitness value
test for termination criterion (based on fitness, on number of generations, etc.)
Until test is positive
End EA

This is a general algorithm and applies to all variants of EA, whether Genetic Algorithms, Evolutionary Programming or Evolution Strategies or other of the kind.

The driving force behind any EA is the selection operator. However, it requires the action of the replication or reproduction operators – recombination and mutation – to generate new points in space to be evaluated. Some EA give relevance to the mutation operator while others relies mostly on the recombination operator.

Mutation is an operator acting on a single individual, particle or chromosome and generates a new point in the search space by modifying the coordinates of a previous point. The definition of the mutation operator may be dependent of the particular problem under analysis. In Evolution Strategies/Evolutionary Programming, where typically (not necessarily) an individual is composed of a string of real variable values, the classical mutation procedure adopts Gaussian or Lognormal random perturbations of each variable. The amplitude of mutation is governed by the variance of the distribution

regulating mutations, and its square root (standard deviation) is often called *learning rate*.

In self-adaptive Evolutionary Algorithms, the learning rates are transformed into variables and added to the chromosome [9,10]. During the process, they are themselves subject to mutation and selection and eventually they acquire values that allow a near optimal progression rate towards the optimum [11]. Self-adaptation was historically used first on the operator mutation.

Recombination generates a new individual by combining features from a set of individuals in the population. For the operator recombination many forms have been proposed. Classical variants are, for example:

Uniform crossover – in this variant, the value for each variable in the newly formed individual is obtained by randomly selecting one of the μ parents to "donate" its value.

Intermediary recombination – in this variant, the value of any variable in the offspring receives a contribution from all parents. This could result either from averaging the values of all parents (global intermediary recombination) or from averaging the values from a subset of the parents only, randomly chosen (local intermediary recombination). In these processes, one may still chose to average values with equal weights or to randomly define weights for a weighted average. In the case of $\mu = 2$, one could have the value of a variable given by

$$x_k^{new} = u_k x_{k,j1} + (1 - u_k) x_{k,j2}$$

where the indices j1 and j2 denote the two parent individuals and u_k is sampled from an uniform distribution in [0,1]. This leads to new individuals in the line segment between the two parents, but the concept may extend to points in the line but external to the segment if u_k is allowed to be sampled in a larger interval.

Point crossover – in this variant, parallel to the one adopted in genetic algorithms, first one randomly defines π crossover points, common to all individuals in the set of parents, and then the offspring successively receives a part from each parent, in turns.

Good schemes of mutation and recombination must assure a convenient exploration of the complete search space, but must also allow exploitation of interesting or promising regions. Achieving a balance between these two desirable features is not at all easy. The self-adaptive EA variants, providing dynamic adaptive change in mutation rates, try to satisfy this double objective. However, although self-adaptation in crossover has been proposed, so far no scheme for a self-adaptive recombination operator has been extensively adopted. A good review is found in [12].

6.3 The Movement Rule of PSO

The classical PSO – Particle Swarm Optimization, conceived by Kennedy and Eberhart [13], relies on the parallel exploration of the search space by a set of solutions or alternatives called "particles" that are successively transformed along the process. From one iteration to the following, each particle $\mathbf{X}_\mathrm{i}$ generates a new one according to a "movement rule", which requires that one keeps record of the best point $\mathbf{b}_\mathrm{i}$ found by the particle in its past life and the current global best point $\mathbf{b}_\mathrm{g}$ found by the swarm of particles in their past life.

The movement rule states that

$$\mathbf{X_i^{new}} = \mathbf{X_i} + \mathbf{V_i^{new}} \tag{6.1}$$

where $\mathbf{V}_\mathrm{i}$ is called the particle i velocity and was classically defined by

$$\mathbf{V}_\mathrm{i}^\mathrm{new} = \mathbf{V}_\mathrm{i} + \mathrm{Rnd}().\mathbf{Wm}_\mathrm{i}(\mathbf{b}_\mathrm{i} - \mathbf{X}_\mathrm{i}) + \mathrm{Rnd}().\mathbf{Wc}_\mathrm{i}(\mathbf{b}_\mathrm{G} - \mathbf{X}_\mathrm{i}) \tag{6.2}$$

where the first term of the summation represents inertia or habit (the particle keeps moving in the direction it had previously moved), the second represents memory (the particle is attracted to the best – past – point in its trajectory) and the third represents cooperation or information exchange (the particle is attracted to the best point found by all particles).

The parameters $\mathbf{W}$ are diagonal matrices with weights fixed in the beginning of the process (index m is for the memory weights and index c is for the cooperation weights). Rnd() are random numbers sampled from a uniform distribution in [0,1]. Figure 6.1 illustrates the concept.

This transformation (movement) scheme proved adequate to make the swarm converge to the zone of the optimum but failed to assure fine tuning in convergence to an accurate optimum position. Therefore, a new scheme has been proposed [11]:

$$\mathbf{V}_\mathrm{i}^\mathrm{new} = \mathrm{Dec(t)}\mathbf{Wi}_\mathrm{i}\mathbf{V}_\mathrm{i} + \mathrm{Rnd}().\mathbf{Wm}_\mathrm{i}(\mathbf{b}_\mathrm{i} - \mathbf{X}_\mathrm{i}) + \mathrm{Rnd}().\mathbf{Wc}_\mathrm{i}(\mathbf{b}_\mathrm{G} - \mathbf{X}_\mathrm{i}) \tag{6.3}$$

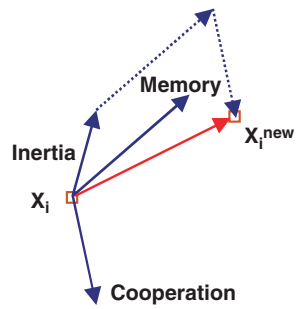

Fig. 6.1. Illustrating the movement of a particle in PSO, influenced by the three terms Inertia, Memory and Cooperation: the final transformation vector from $\mathbf{X}_\mathrm{i}$ to $\mathbf{X}_\mathrm{i}^\mathrm{new}$ is composed of three contributions

where Dec(t) is a function decreasing with the progress of iterations, reducing progressively the importance of the inertia term, and new weights **Wi** are included.

Another version for PSO has been proposed by Maurice Clerc in [14], with the introduction of a *constriction factor* K, applied to each dimension of the particle space. In this version, the movement rule is

$$\mathbf{V}_{\mathrm{i}}^{\mathrm{new}} = \mathbf{K}\left[\mathbf{V}_{\mathrm{i}} + \mathrm{Rnd}().\mathbf{Wm}_{\mathrm{i}}(\mathbf{b}_{\mathrm{i}} - \mathbf{X}_{\mathrm{i}}) + \mathrm{Rnd}().\mathbf{Wc}_{\mathrm{i}}(\mathbf{b}_{\mathrm{G}} - \mathbf{X}_{\mathrm{i}})\right] \tag{6.4}$$

where **K** is a diagonal matrix of constriction factors. For dimension k, it is given by

$$\mathrm{K}_{\mathrm{k}} = \frac{2}{\left|2 - \mathrm{W}_{\mathrm{k}} - \sqrt{\mathrm{W}_{\mathrm{k}}^2 - 4\mathrm{W}_{\mathrm{k}}}\right|} \text{ with } \mathrm{W}_{\mathrm{k}} = \mathrm{Wm}_{\mathrm{k}} + \mathrm{Wc}_{\mathrm{k}} \text{ and } \mathrm{W}_{\mathrm{k}} > 4 \tag{6.5}$$

As one can see, none of these mechanisms is really adaptive and certainly not self-adaptive. Therefore, PSO is not an Evolutionary method. In fact, it does not rely on a selection operator as its driving force. It depends on *movement rule* that allows the generation of new individuals in space and this rule is such that, by itself, pushes the search towards the optimum.

6.4 EPSO as an Evolutionary Algorithm

6.4.1 Recombination in EPSO

In 2002 we have proposed an algorithm called EPSO [1] where instead of using the classical mutation and recombination operators to produce new individuals we have adopted the general scheme of the movement rule of PSO.

The general scheme of the movement rule is (whatever the variant)

$$\mathbf{V}_{\mathrm{i}}^{(\mathrm{k}+1)} = \mathbf{A}\mathbf{V}_{\mathrm{i}}^{(\mathrm{k})} + \mathbf{B}\left(\mathbf{b}_{\mathrm{i}} - \mathbf{X}_{\mathrm{i}}^{(\mathrm{k})}\right) + \mathbf{C}\left(\mathbf{b}_{\mathrm{G}} - \mathbf{X}_{\mathrm{i}}^{(\mathrm{k})}\right) \tag{6.6}$$

If we examine this scheme, we conclude that a new particle $\mathbf{X}_{\mathrm{i}}^{(\mathrm{k}+1)}$ is formed as a combination of four other points:

- Its direct ancestor $\mathbf{X}_{\mathrm{i}}^{(\mathrm{k})}$
- The ancestor of its ancestor $\mathbf{X}_{\mathrm{i}}^{(\mathrm{k}-1)}$
- A (possibly) distant past best ancestor $\mathbf{b}_{\mathrm{i}}$
- The current global best of the swarm $\mathbf{b}_{\mathrm{G}}$

We can give a different aspect to the movement rule:

$$\mathbf{X}_{\mathrm{i}}^{(\mathrm{k}+1)} = \mathbf{X}_{\mathrm{i}}^{(\mathrm{k})} + \mathbf{A}\left(\mathbf{X}_{\mathrm{i}}^{(\mathrm{k})} - \mathbf{X}_{\mathrm{i}}^{(\mathrm{k})}\right) + \mathbf{B}\left(\mathbf{b}_{\mathrm{i}} - \mathbf{X}_{\mathrm{i}}^{(\mathrm{k})}\right) + \mathbf{C}\left(\mathbf{b}_{\mathrm{G}} - \mathbf{X}_{\mathrm{i}}^{(\mathrm{k})}\right) \tag{6.7}$$

$$\mathbf{X}_{\mathrm{i}}^{(\mathrm{k}+1)} = (1 + \mathbf{A} - \mathbf{B} - \mathbf{C})\mathbf{X}_{\mathrm{i}}^{(\mathrm{k})} - \mathbf{A}\mathbf{X}_{\mathrm{i}}^{(\mathrm{k}-1)} + \mathbf{B}\mathbf{b}_{\mathrm{i}} + \mathbf{C}\mathbf{b}_{\mathrm{G}} \tag{6.8}$$

and we realize now that the sum of the parameters multiplying the four contributors to generate the offspring is equal to 1. It is therefore tempting to identify this expression with an intermediary recombination in EA with $\mu = 4$ and a special rule to determine who the parents are (they are not randomly selected). This means that we are considering an *enlarged population* including not only the active particles but also the direct ancestors and the set of the past best ancestors.

It is a recombination rule that has the remarkable property of pushing the population towards the optimum, as the PSO algorithms have demonstrated. Therefore, if joined together with a selection mechanism, which also pushes the population towards the optimum, one may expect that some cumulative effect may improve the performance of an optimizing algorithm.

6.4.2 Self-Adaptive Recombination

To determine the best values to use in **A**, **B** and **C**, EPSO relies on a self-adaptive mechanism. These parameters, considered as strategic, will be subject to selection and hopefully will evolve to values adapted to the type of landscape being searched.

Given a population with a set of particles, the general scheme of EPSO is the following:

- REPLICATION – each particle is replicated r times
- MUTATION – each particle has its strategic parameters mutated
- REPRODUCTION – each mutated particle generates an offspring through recombination, according to the particle movement rule, described below
- EVALUATION – each offspring has its fitness evaluated
- SELECTION – by stochastic tournament or other selection procedure, the best particles survive to form a new generation, composed of a selected descendant from every individual in the previous generation

Mutation of a parameter w into w^* is ruled by multiplicative Lognormal random numbers such as in $w_i^* = w_i[\log N(0,1)]^\tau$ or by additive Gaussian distributed random numbers such as in $w_i^* = w_i + \sigma N(0,1)$. The learning parameter (τ or σ) must be fixed externally.

The recombination rule for EPSO becomes (see Figure 6.2):

$$\mathbf{X}_i^{(k+1)} = \mathbf{X}_i^{(k)} + \mathbf{V}_i^{(k+1)} \tag{6.9}$$

$$\mathbf{V}_i^{(k+1)} = w_{i1}^* \mathbf{V}_i^{(k)} + w_{i2}^*(\mathbf{b}_i - \mathbf{X}_i) + w_{i3}^*(\mathbf{b}_G^* - \mathbf{X}_i)\,\mathbf{P} \tag{6.10}$$

where:

$\mathbf{b}_i$ – best point found by particle i in its past life up to the current generation

$\mathbf{b}_G$ – best overall point found by the swarm of particles in their past life up to the current generation

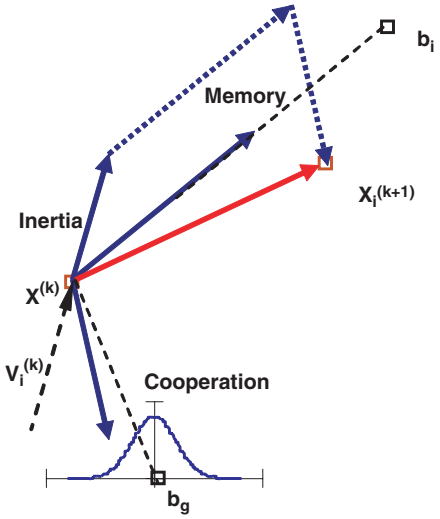

Fig. 6.2. Illustration of the EPSO movement rule. Notice that the vector associated with the *cooperation* factor does not point exactly to the global optimum b_g but to a mutated location

$\mathbf{X}_i^{(k)}$ – location of particle i at generation k
$\mathbf{V}_i^{(k)} = \mathbf{X}_i^{(k)} - \mathbf{X}_i^{(k-1)}$ – is the velocity of particle i at generation k
w_{i1} – weight conditioning the *inertia* term
w_{i2} – weight conditioning the *memory* term
w_{i3} – weight conditioning the *cooperation* or *information exchange* term
$\mathbf{P}$ – communication factor (discussed below)

The symbol * indicates that the parameter will undergo mutation. In the most effective EPSO variant, not only the weights affecting the components of movement are mutated but also the global best $\mathbf{b}_G$ is randomly disturbed to give

$$\mathbf{b}_G^* = \mathbf{b}_G + w_{i4}^* N(0,1) \tag{6.11}$$

where w_{i4} is the forth strategic parameter (target weight) associated with particle i. It controls the "size" of the neighborhood of $\mathbf{b}_G$ where it is more likely to find the real global best solution.

Selection is modeled from the Stochastic Tornament concept: among the ofspring of each particle, one compares the best one with another particle randomly sampled, and the best is selected with probability $(1-\text{luck})$, where the *luck* parameter is defined in [0,1] but is usually small. If luck = 0 we have elitist selection.

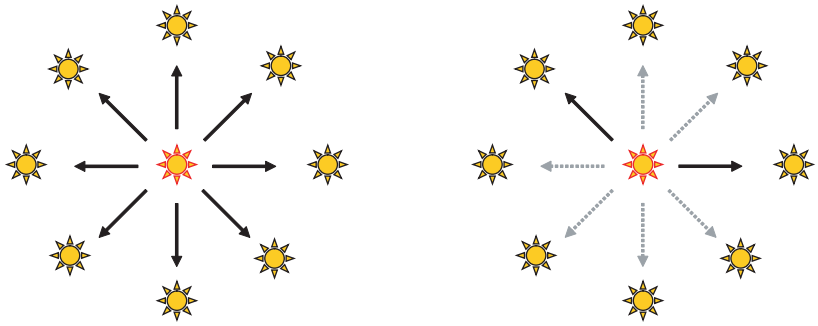

Fig. 6.3. Illustration of the star communication topology (left) and the stochastic star topology (right). In this latter case, the information of the location of the global best is only communicated to a subset of the swarm (thick arrows) and the composition of this subset varies from iteration to iteration

6.4.3 Communication Topology

The communication factor **P** induces a *stochastic star* topology for the communication among particles (Figure 6.3). It is a diagonal matrix affecting all dimensions of an individual, containing binary variables of value 1 with probability p and value 0 with probability (1-p); the p value controls the passage of information within the swarm and is 1 in classical formulations (this is the *star*).

This stochastic scheme oscillates between the star arrangement and a selfish version called *cognitive model* in [15], where no communication exists and a descendent of an individual is built only of contributions from its ancestor line.

Experimental results have suggested that a communication probability of $p = 0.20$ leads in many cases to better results than a classical deterministic star model with $p = 1$. One is lead to believe that restraining the free flow of information about the global best allows more local search by each particle and avoids premature convergence. As it is easily observed, this is yet another way of acting on the recombination operator.

The importance of not having a fixed communication topology in PSO has been suggested by works reported such as in [16] or [17]. Truly adaptive schemes for communication topology are yet to be proposed.

6.4.4 EPSO as a PSO

Because EPSO adopts the movement rule of PSO, we can always look at how it works as a swarm whose movement is ruled by weights that self-adapt in order to produce a global drift more adjusted to the landscape. Furthermore, because these weights are subject to mutation, this may give an extra chance for the swarm to escape local minima (i.e., having particles that still explore other regions of space, because they may gain enough speed).

On the other hand, EPSO also shows ability to focus and zoom in the optimum, precisely because mutations in the weights may favor the selection of the cooperation factor and reduce the importance of inertia and memory, if this strategy proves successful. This may in part explain why EPSO has shown, in many tests, robustness by consistently reaching the same optimum in a number of runs.

Lessons learned from PSO surely play a decisive role in improving EPSO algorithms. However, the point of view of EPSO as an evolutionary algorithm with self-adaptive recombination may give greater insight on how the algorithm successfully works and allow greater degree of freedom in designing better variants without being stuck with the rigid classical PSO form. One such example is described in the following section.

6.4.5 dEPSO: The Differential Evolution Version of EPSO

Differential Evolution [18], an evolutionary algorithm that uses a recombination rule that resembles some characteristics of the movement rule of PSO, has come up in the literature as an alternative method whose users claim that it displays good efficiency in reaching an optimum. There is more than a vague similarity between PSO and DE, In its variant DE2, from a particle $\mathbf{X}^{(\mathbf{k})}$ at iteration k a new individual is produced through

$$\mathbf{X}_{\mathrm{i}}^{(\mathrm{k}+1)} = \mathbf{X}_{\mathrm{i}}^{(\mathrm{k})} + \mathrm{w}_{\mathrm{i}2}(\mathbf{X}_{\mathrm{A}\mathrm{i}} - \mathbf{X}_{\mathrm{B}\mathrm{i}}) + \mathrm{w}_{\mathrm{i}3}(\mathbf{b}_{\mathrm{G}} - \mathbf{X}_{\mathrm{i}}) \tag{6.12}$$

where $\mathbf{X}_{\mathrm{A}}$ and $\mathbf{X}_{\mathrm{B}}$ are two different individuals sampled from the population and distinct from the current individual $\mathbf{X}$ and $\mathbf{b}_{\mathrm{G}}$ is the global best. In this recombination operator, the attraction for the particle past best is replaced by the difference between to other individuals.

The general principle of DE relies on the perception by the algorithm of the topology of the function being optimized by sensing macro-gradients through differences of points. In a way, this is exactly what a PSO algorithm also does.

We can add this flavor to EPSO by changing the movement rule precisely in the memory element, in a version we will call dEPSO. Instead of using the difference between a particle and its past best, we will use in this term the difference between two particles A and B randomly selected among the population (this can be the enlarged population including the set of particles past bests). The movement rule of dEPSO becomes

$$\mathbf{V}_{\mathrm{i}}^{(\mathrm{k}+1)} = \mathrm{w}_{\mathrm{i}1}^{*}\mathbf{V}_{\mathrm{i}}^{(\mathrm{k})} + \mathrm{w}_{\mathrm{i}2}^{*}(\mathbf{X}_{\mathrm{A}\mathrm{i}} - \mathbf{X}_{\mathrm{B}\mathrm{i}}) + \mathrm{w}_{\mathrm{i}3}^{*}\left(\mathbf{b}_{\mathrm{G}}^{*} - \mathbf{X}_{\mathrm{i}}\right)\mathbf{P} \tag{6.13}$$

It is obvious that the communication topology becomes more complex, because a particle must now be aware not only of the location of the global best but also of the location of two other particles. These two particles may be chosen among the ones in the current restrict swarm or chosen among the enlarged swarm, including the set of the past best locations of all particles.

6.4.6 The Enhanced Elitist Version of EPSO

Closely observing the EPSO scheme, we become aware that that during the evolution of the swarm, selection acts only on the offspring and never on the set of parents plus offspring (in Evolution Strategies language, it is a comma strategy and not a plus strategy). To introduce a plus strategy and limited elitism, we have introduced changes in EPSO by dealing with the particle that finds the global best in a different way to the rest of the swarm: the particle at the global best is not eliminated unless one of its descendents finds a better point and replaces it – it competes with its offspring and pure elitism applies, not stochastic tournament.

Furthermore, for this particle we generate not just r descendents but a set $d > r$ – a local mini-swarm – and the mutations of the weights are done with a much narrower variance.

This variant introduces a new concept: that not all particles should be treated the same in the swarm. This reasoning opens a new path for the development of more flexible and (hopefully) more efficient variants of EPSO.

6.5 Experiments with EPSO

6.5.1 Test Functions

We now present some experiments we've designed to test the basic version of EPSO [equations (6.9) and (6.10) with P = 1] against the standard PSO 2006 by M. Clerc [19]. These experiments were based on four well-known test functions: the first and fourth functions represent difficult optima to be found, the third is the classical function that was used by ES researchers such as Beyer [11] to define the concepts of progress rate and the second exhibits a large number of deceiving local optima. All these functions describe unconstrained real-valued optimization problems.

Schaffer's function: $$\min \mathrm{f}_1(\mathrm{x}) = 0.5 + \frac{\left(\sin\sqrt{\mathrm{x}^2+\mathrm{y}^2}\right)^2 - 0.5}{(1.0 + 0.001(\mathrm{x}^2+\mathrm{y}^2))^2}$$

Alpine function: $$\max f_4(x) = \sin(x_1)\ldots\sin(x_n) \times \sqrt{x_1 \ldots x_n}$$

Sphere function: $$\min f_3(x) = \sum_{i=1}^{n} (x_i - 1)^2$$

Rosenbrock function: $$\min \mathrm{f}_2(\mathrm{x}) = \sum_{\mathrm{i}=1}^{\mathrm{n}} (100 \times \left(\mathrm{x}_{\mathrm{i}+1} - \mathrm{x}_{\mathrm{i}}{}^2\right)^2 + \mathrm{x}_{\mathrm{i}} - 1)^2)$$

The parameters used in the tests are presented in Table 6.1. The threshold used as the stopping criterion is listed in the column on the right; this threshold has been defined as a value close to the exact optimum, which is known for all problems.

6.5.2 Tests and Results

The first battery of tests defined was [1]:

1. In 500 runs randomly initiated, determine the average number of fitness evaluations needed to reach the demanding stopping threshold defined in Table 6.1.
2. Determine the average result offered by both algorithms, for a fixed number of fitness evaluations, in 500 runs randomly initiated. The limit was set at 200,000 evaluations, corresponding to 20 particles over 5,000 iterations in EPSO and 20 particles over 10,000 iterations in PSO (to have the same number of function evaluations).

The results of the tests are summarized in Tables 6.2 and 6.3.

– EPSO clearly displayed superiority over PSO in the 2-dimension Schaeffer's function. Figure 6.4 displays a typical comparison of convergence behavior between EPSO and classical PSO. Notice that the scale for the fitness (or objective) function is logarithmic.

Table 6.1. Parameters used in the test functions

Function	n	Domain	Stopping threshold
f_1 – Schaffer	2	$[-50, 50]^n$	0.0001
f_2 – Alpine	2	$[0, 100]^n$	98.9627
f_3 – Sphere	30	$[0, 100]^n$	0.01
f_4 – Rosenbrock	30	$[0, 30]^n$	1

Table 6.2. Average number of evaluations to reach the stopping threshold

Function	EPSO	PSO
$f1$ – *Schaffer 2-dimension*	4087	11165
f_2 – *Alpine 2-dimension*	16421	17007
f_3 – *sphere 30-dimension*	9789	26187
f_4 – *Rosenbrock 30-dimension*	28452	131462

Table 6.3. Average result for 200 000 evaluations in 500 runs, except for $f1$ where the average result is for 500 runs of 10 000 evaluations

Function	EPSO	PSO
$f1$ – *Schaffer 2-dimension*	1.65E-06	1.06E-03
f_2 – *Alpine 2-dimension*	98.9627	98.8484
f_3 – *sphere 30-dimension*	5.28E-09	4.40E-02
f_4 – *Rosenbrock 30-dimension*	1.91E-04	1.4243

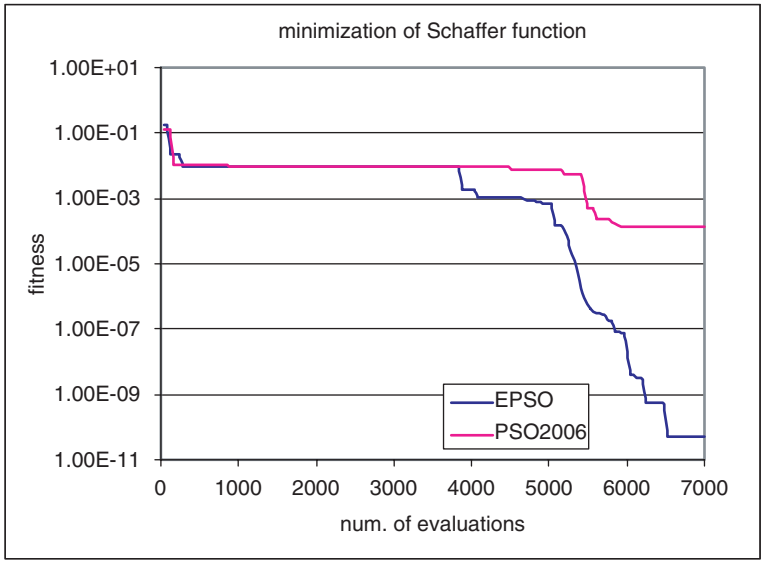

Fig. 6.4. Typical convergence of EPSO and classical PSO for Schaffer's function [1]

The Alpine function has many local optima and is therefore a good test for the ability of an algorithm to escape being trapped and instead discover the global optimum. Our first set of tests has been with the Alpine function defined in a 2-dimension space, and once more EPSO clearly displayed superiority over classical PSO. Figure 6.5 displays a typical comparison of convergence behavior between EPSO and classical PSO.

In another experiment, we used the algorithm to find the optimal maximum of the Alpine 10-dimension function in a $[0, 100]^{10}$ search space. In Figure 6.6 we can observe the average convergence in an experiment series of 20 simulations, with 40 particles. Once again we verify a superior behavior of EPSO, compared to the standard PSO. This has happened for all spaces of no matter how many dimensions.

The spherical model has been used extensively in Evolution Strategies to help in defining concepts such as "rate of progress" – as the statistical expectation of the improvement in the fitness value from any iteration to the following. Because it is an isotropic model, the improvement from iteration to iteration in an evolutionary process could be represented simply by a scalar function of the distance to the optimum. In this case, we are before a problem with a single optimum and we can test therefore the relative efficiency of EPSO against classical PSO. EPSO clearly displayed superiority over PSO in a 30-dimension sphere, and our interpretation for this fact is based on the explorative capacity conferred to EPSO by both the self-adaptive changes in the weights and on the definition of a foggy target instead of aiming the particles at the best-so-far point.

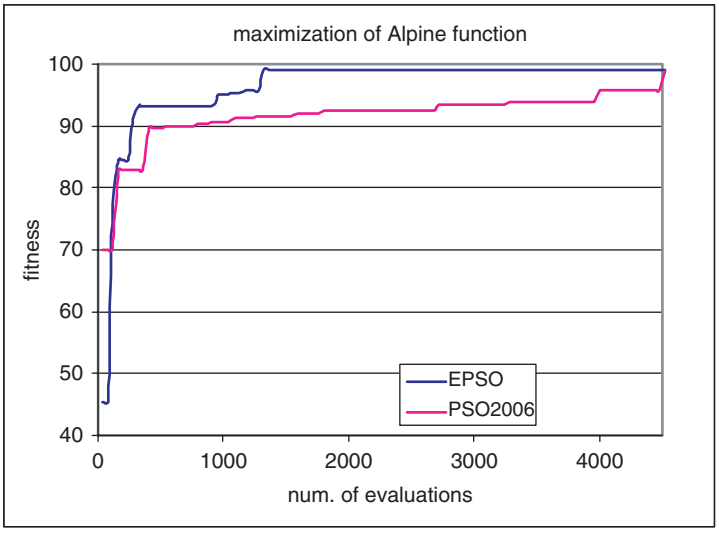

Fig. 6.5. Typical convergence of EPSO and classical PSO for the 2-dimension Alpine function [1]

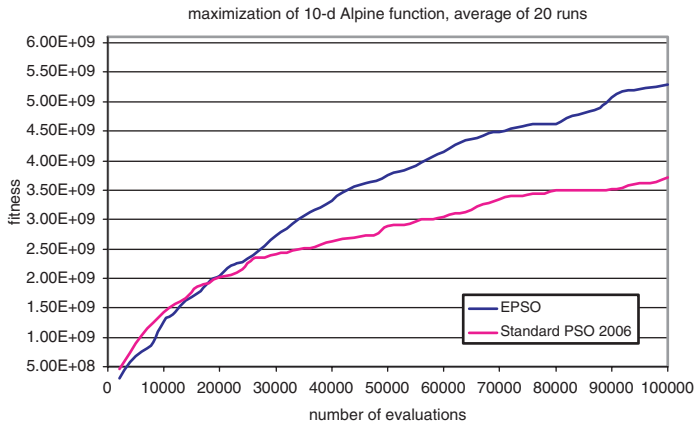

Fig. 6.6. Comparison of the average fitness evolution in EPSO and standard PSO for the 10-dimension Alpine function

Figure 6.7 displays a typical comparison of convergence behavior between EPSO and PSO. Notice again that the scale for the fitness (or objective) function is logarithmic.

We have also analyzed the effect of the variation of population size in this case. In 500 runs as above, for populations of 5, 10, 15 and 20 particles or individuals, the percentage of runs that did not satisfy a demanding threshold criterion has been registered. Figure 6.8 displays the results along

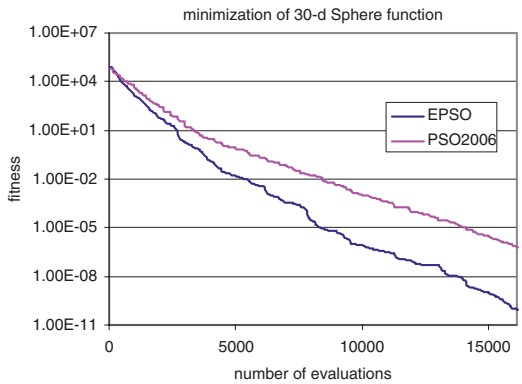

Fig. 6.7. Typical convergence of EPSO and standard PSO for the Sphere function [1]

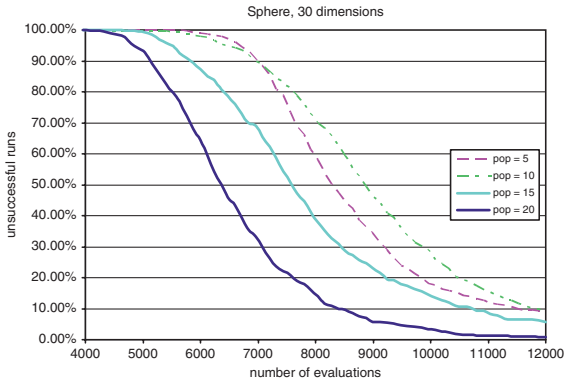

Fig. 6.8. Average error of the EPSO algorithm for different populations, measured by the percentage of runs that did not meet a given threshold close to the optimum

many iterations – it shows that enlarging the population provides a net benefit measured in the number of fitness function evaluations and suggests that in this case a population larger than 20 particles would perhaps lead to an even smaller computing effort. For a 10% failure rate, one gets below the defined threshold close to the optimum in approximately 200 EPSO iterations, which is remarkable.

The Rosenbrock function has a global optimum at point (1,. . . ,1) with value 0 but the basin of attraction is extremely narrow and the path to reach it a deep valley, due to the twisted topology of the function in its vicinity, making it difficult to reach. It serves as a test for convergence in difficult circumstances. EPSO clearly displayed superiority over PSO in this function, defined in a 30-dimension space. Figure 6.9 displays a typical comparison of convergence behavior between EPSO and standard PSO. Notice again that the scale for the fitness (or objective) function is logarithmic. A similar comparison was

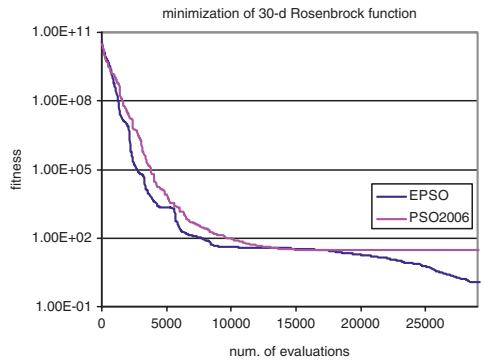

Fig. 6.9. Typical convergence of EPSO and classical PSO for Rosenbrock's function [1]

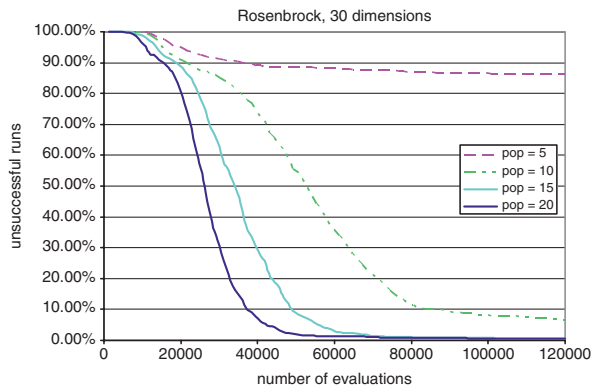

Fig. 6.10. Average error of the EPSO algorithm for different populations, measured by the percentage of runs that did not meet a given threshold close to the optimum

presented in [1] and relates to early experiments. It happens that the quality of the random number generator has shown to be important in the performance of these kinds of algorithms. Nevertheless the comparison in qualitative terms remains valid.

We presently have an efficient computer code implementing EPSO that is able to always reach the optimum of the Rosenbrock function although requiring a bit more than 40.000 fitness function evaluations for a 30 dimension space to do so.

We have also in this case analyzed the effect of the variation of population size. In 500 runs as above, for populations of 5, 10, 15 and 20 particles or individuals, the percentage of runs that did not satisfy a demanding threshold criterion has been registered.

Figure 6.10 displays the results along many iterations; one may clearly see that a population of 5 has a significantly worse behavior. The evolution in the

graph and its comparison with Figure 6.8 leads us to speculate that the easier nature of the Sphere problem allows its solution by a small population, while the hard nature of the Rosenbrock function demands more exploratory power and therefore a larger population is needed.

6.5.3 Communication Probability p in the Rosenbrock Function

The effect of the communication probability p in forming a stochastic star communication scheme can be well observed in the Rosenbrock function in a space of D = 30, with a swarm of 20 particles, additive mutation with $\sigma = 0.2, r = 2$ and a maximum effort of 50,000 fitness function evaluations.

The communication probability was altered while all other parameters were left unchanged, Figure 6.11 displays the results obtained on an average of 20 runs. A number of things are immediately obvious:

- There is an optimum p value for finding the best result (closest to the optimum). The figure suggests that it may be approximately 0.75 in this case.
- There is an optimum p value that assures the best robustness to the algorithm (meaning that one has minimum variance in results for repeated runs).
- Average and robustness dependence on p are highly correlated – which is a good thing. This means that when choosing the p that leads on average to the best result, one also assures the strongest robustness in EPSO runs – which is important to reassure users that they can have confidence in the result when they perform a single run.

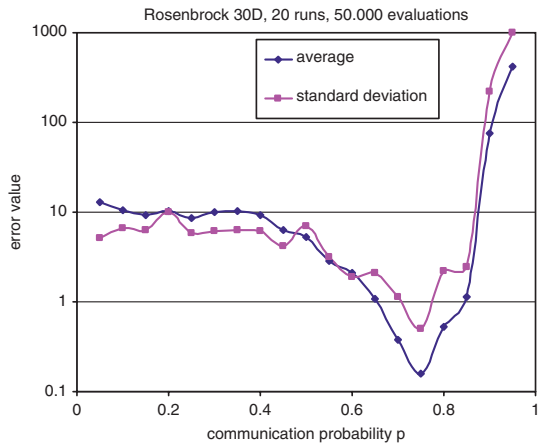

Fig. 6.11. Average error and error variance in 20 runs for the 30-dimension Rosenbrock function as a function of the communication probability p in the stochastic star

6.5.4 The Enhanced Elitist Version of EPSO

The difficulty with problems such as the Rosenbrock function is that during the evolution of the swarm, selection acts only on the offspring and never on the set of parents plus offspring (en Evolution Strategies language, it is a comma strategy and not a plus strategy). To introduce a plus strategy and limited elitism, we have introduced changes in EPSO by dealing with the particle that finds the global best in a different way to the rest of the swarm: the particle at the global best is not eliminated unless one of its descendents finds a better point and replaces it – it competes with its offspring and pure elitism applies, not stochastic tournament.

Furthermore, for this particle we generate not just r descendents but a set $d > r$ – a local mini-swarm – and the mutations of the weights are done with a much narrower variance.

Applying this elitist version to the Rosenbrock function, with the same data and $r = 2$ but $d = 4$ and $\tau = 0.005$ just for the mini-swarm, we have obtained better results than with the standard EPSO with a single main swarm. We are not presenting results because the ones available were obtained with an earlier version of EPSO and cannot be compared with the results in the previous section.

This represents an interesting improvement and suggests that exploring an elitist strategy together with the stochastic star communication topology is a way to improve the performance of EPSO in problems with narrow and difficult valleys such as is the case of the Rosenbrock function. More research is expected in this topic.

6.5.5 The Differential Evolution Version: dEPSO

The effectiveness of this model is tested in the following problem of unit commitment in power systems: given a set of generators and their generation cost curves, define which generators should be shut down and which should be in service and at which loading level, in order to minimize the overall cost (start up cost plus operation cost).

The problem is complex because of the cost functions for generators that must be considered – a mixed-integer non-linear program. Because of technical limits, the domain of a generator is not connected – there is a point (0,0) corresponding to generator shut down and then there is a gap until a point $(P_{min}, c(P_{min}))$ corresponding to the technical minimum of the machine. And the general shape of the cost functions implies that the problem has a non-convex nature – therefore, many local optima may appear.

The comparative performance of dEPSO is tested in a particular problem. The data are:

- the number of generators – ngen = 5

– the parameters of the cost function of each generator – this function is assumed to be a cubic polynomial, with 4 parameters a_i for i = 1 to 4:

$$C = a_0 + a_1 P + a_2 P^2 + a_3 P^3$$

where C is the generation cost in \$/hour and P is the generator output in MW.
– the technical minimum and maximum of each generator P_{min} and P_{Max}
– the load, located at a single bus (transmission system neglected) – L = 15 MW

Cost curves and technical limits given in Table 6.4.

The objective is to minimize the sum of the costs for the five generators, noting that the domain of each variable is not continuous. The EPSO model demands an individual defined by its object parameters (5 variables) and strategic parameters (4 weights). We used Lognormal mutations with $\tau = 1$, 20 particles, r = 2 and 1000 generations. Initial values of the weights were set to 0.5 except for the target weight where the value was set to 0.001.

The cost curves are depicted in Figure 6.12 as well as the optimal solution, which is numerically presented in Table 6.5.

Table 6.4. Coefficients for the cubic cost polynomial function for each generator

Generator	a_0	a_1	a_2	a_3	Pmin	Pmax
g1	1	0,5	0,1	0,03	0 or 1	10
g2	2	0,4	0,2	0	0 or 2	10
g3	4	0,3	0,3	0	0 or 7	10
g4	6	1,5	0,15	0	0 or 2	10
g5	0	4	0	0	0 or 1	10

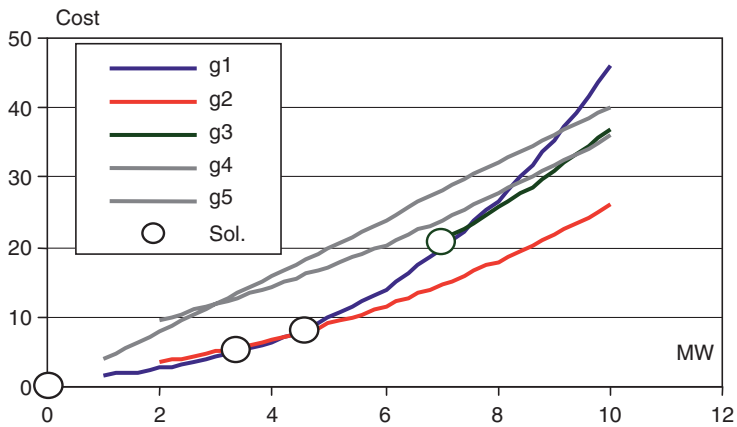

Fig. 6.12. Cost curves for the unit commitment problem and the best solution with 3 generators dispatched and 2 shut down

Table 6.5. Unit commitment and power dispatch for the optimal solution

g1	g2	g3	g4	g5	Cost
3.414	4.586	7	0	0	33.9068

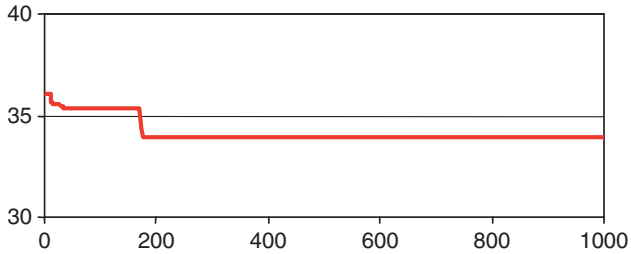

Fig. 6.13. Evolution of the best fitness during 1000 generations in one run of the EPSO algorithm

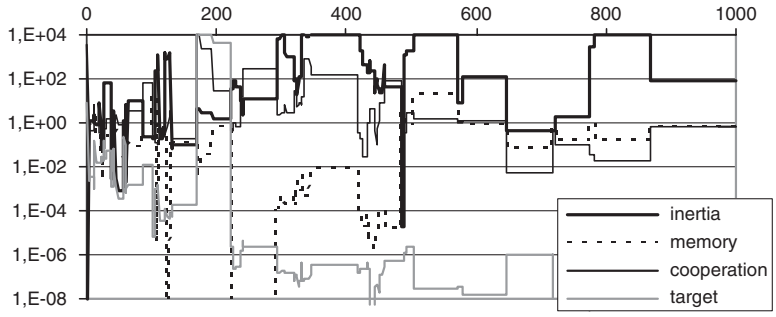

Fig. 6.14. Evolution of weights of the best particle at each iteration during the unit commitment optimization (1000 generations) using EPSO

Figure 6.13 displays the evolution of best fitness during an EPSO run and Figure 6.14 shows the weights of the global best particle kept at each iteration, for the same run. The best particle pattern shows that improvements were made till late in the process, in fact tiny adjustments after the big jump before generation 200, after which the perturbation in the target weight faded away. In the last generations, we also see that the inertia weight was dominant over the other weights.

Figure 6.15 shows the fitness evolution for a dEPSO algorithm and in Figure 6.16 shows the evolution of weights of a single specific particle in the optimization process. It suggests that the selection procedure indeed distinguished among the roles of the different components of the movement rule.

These are success cases, but how do the EPSO and dEPSO versions compare? Table 6.6 allows us to examine the best value found and the average in

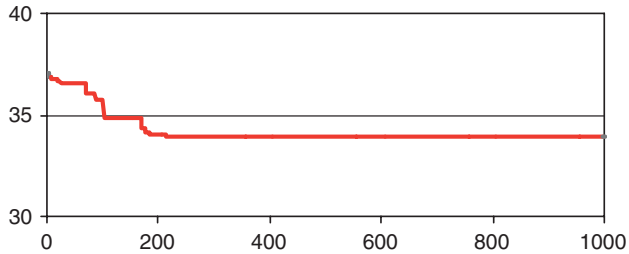

Fig. 6.15. Evolution of the best fitness during 1000 generations in one run of the dEPSO algorithm

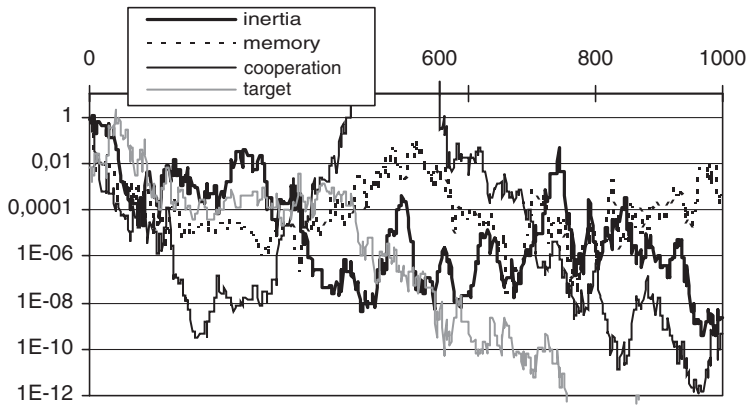

Fig. 6.16. Evolution of weights of a particle during the unit commitment optimization (1000 generations) using dEPSO

Table 6.6. Comparison of the performance of EPSO and dEPSO

	Av. 20 runs	Best found	No. times opt. found
EPSO	34,81575	33,90696	7/20
dEPSO	34,07073	33,90683	18/20

20 runs and how many times an algorithm discovers the optimum value or the optimum neighborhood, in 1000 generations, in 20 runs.

We can see that dEPSO provided a better average result and, especially, found the location of the global optimum more times than the simple EPSO. This result suggests that the dEPSO variant deserves to be looked at seriously and new research is suggested. The fact is that the problem of Unit Commitment is one very important problem in Power Systems for the economical implications it has and efficient solutions are still being sought that may compete with classical dynamic programming or branch and bound approaches, that have the known limitations of the curse of dimensionality for real systems.

6.6 Optimizing Market Competition

Comparisons of the performance of EPSO with other algorithms have been made in demanding Power System problems by other authors. In [20] the authors compared EPSO with PSO and their adaptive variant of called APSO in co-generation plant operation optimization, with the advantage of EPSO. In [21] one finds a comparison of several methods in voltage stability assessment where EPSO also emerged as the winner.

In this section we will refer to a competition among algorithms that we organized in a very demanding problem, extremely suited for meta-heuristics, given that no analytic solution may be found. It is the case of a simulation of a retail market in energy distribution, where the general objective is to discover a market strategy that will give advantage to a market actor (energy retailer) in a competitive environment.

The simulation was set over the open source intelligent agent platform JADE, which is FIPA compliant.

We will not describe this platform here but a description may be found in [22]. The simulation is composed of 19 agents running in a parallel arrangement of 5 PCs, as shown in Figure 6.17.

Each agent has an internal logic that commands its decisions, which are influenced by load forecasts, network development, reaction of consumers, prices, investment decisions, regulatory constraints, etc. Agents are assumed independent and from their interaction a complex behavior emerges that determines a dynamic market equilibrium while the simulation develops along time.

The agents represented in the platform are in Table 6.7.

One special type of agent is the retailer, which is equipped with a mechanism to predict the behavior of competitors in price setting. But the most unique feature of this agent is that, during the simulation, it performs regularly internal simulations of the market in order to try to optimize its policy for the next moves, acting on variables such as price of energy or money allocated to advertising, increasing company efficiency or investment.

This environment is dynamic because we have equipped the consumer agents with non-linear mechanisms that simulate their response to market conditions. Also, retailers depend on the action of delivery companies, because these agents make decisions about expanding their networks (having a GIS-type representation of the territory) and therefore potentially reaching new consumers.

This is an extremely complex environment to test a number of alternative meta-heuristics to optimize market strategy in the internal simulation of a retailer and to examine which agent (equipped with which algorithm) comes out as a winner (best profits) in a simulation extending for 2 years on a daily basis. We have reported some results in [23, 24] and in this paper we add comparisons including classical PSO, together with EPSO and 3 versions of Genetic Algorithms.

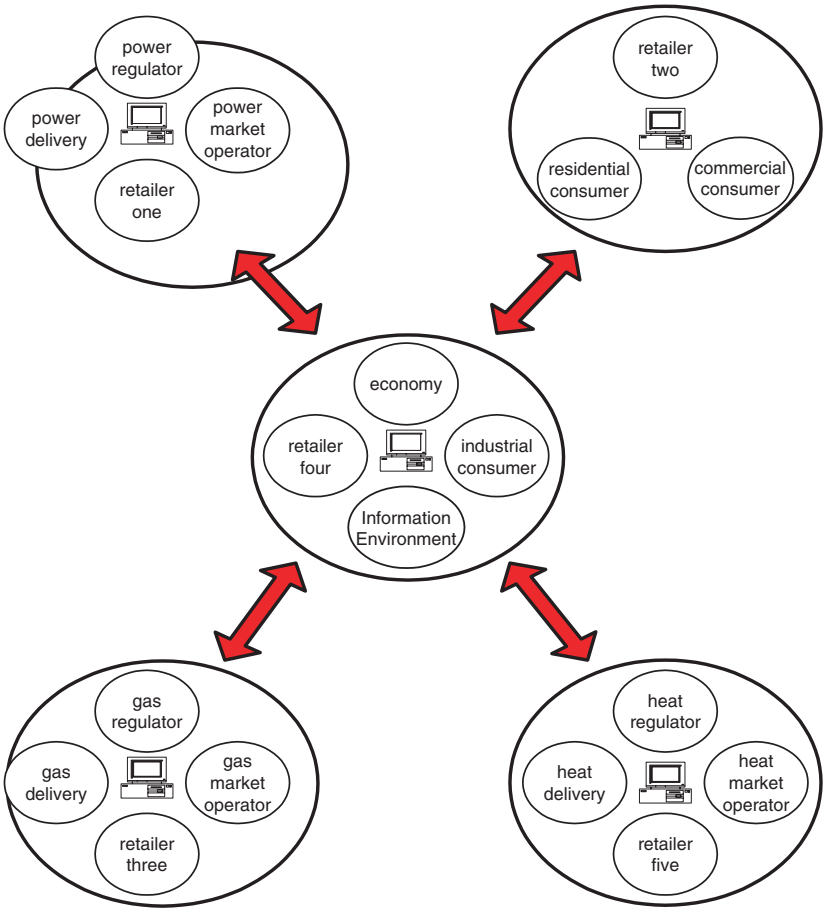

Fig. 6.17. Parallel arrangement of 19 Agents in a cluster of 5 PCs

Table 6.7. Comparison of the performance of EPSO with different values of p and local mini-swarm

AGENTS	
1 Residential cons. group	1 Commercial cons. group
1 Industrial cons. group	2 Electricity Retail Suppl.
2 Gas Retail Suppliers	1 Heat Retail Supplier
1 Power Delivery comp.	1 Gas Delivery company
1 Heat Delivery company	1 Electricity Regulator
1 Electr. Market operator	1 Gas Market operator
1 Heat Market operator	1 Gas Regulator
1 Heat Regulator	1 Economy agent
1 Information Environment	

The Genetic Algorithm versions were:

SSGA – Steady State Genetic Algorithm: a scheme where for a generation to the following we allowed the replacement of only 80% of the population, keeping the best 20%. It adds elitism to the selection, aiming at preserving in the population the most promising individuals so that they may have more chances of combining their good genes with other individuals.

DCGA – Genetic Algorithm with Deterministic Crowding: Deterministic Crowding is a selection technique where similar individuals in the population are paired before comparisons are made, and elitist selection acts on each pair formed. It aims at preserving diversity in the population.

MPGA – Multi-Population Genetic Algorithm: a scheme where we used two sub-populations, each running a SSGA, but exchanging two randomly selected individuals from one of the sub-populations to another before crossover took place. It also aims at preserving diversity.

The PSO algorithm used was the classical formulation with decreasing function in the inertia weight. The EPSO version was the basic one with Lognormal mutations.

The simulation length was of 24 months for market simulation and of 2 months for retailer internal simulations inside the evolutionary process.

The stopping criterion was the same for all algorithms: when performing the first internal simulation, the evolutionary process was stopped if there were no improvement in the fitness function after 50 consecutive generations; in all the following internal simulations, during the market simulation of 24 months, the limit was 10 generations.

In order to compensate for the influence of random events, we run every simulation 5 times. During market simulation, only one retailer was equipped with one type of EA and other retailers were not optimizing; also the investment function of Delivery agents was disconnected to avoid introducing noise in the simulation and disturb the interpretation of results.

The fitness function was the same in all cases: maximizing profit of the retailer.

In a first test, we have fixed the population for all methods as 20 individuals.

Figures 6.18 and 6.19 show the weekly profits made by a retailer agent using PSO and EPSO in five runs. One may observe that EPSO displays robustness in the results. Figure 6.20 compares the performance of the 5 algorithms and we see that EPSO was the clear winner and MPGA is second best while DCGA was the clear loser.

However, one of the possible reasons of bad performance of some algorithms could be early termination because of the application of the stopping criterion. We see clearly in Figure 6.21 that the distribution of computing effort was quite unbalanced. In an attempt to have a fair comparison, we have defined an Experiment 2 and changed the size of populations such that EPSO

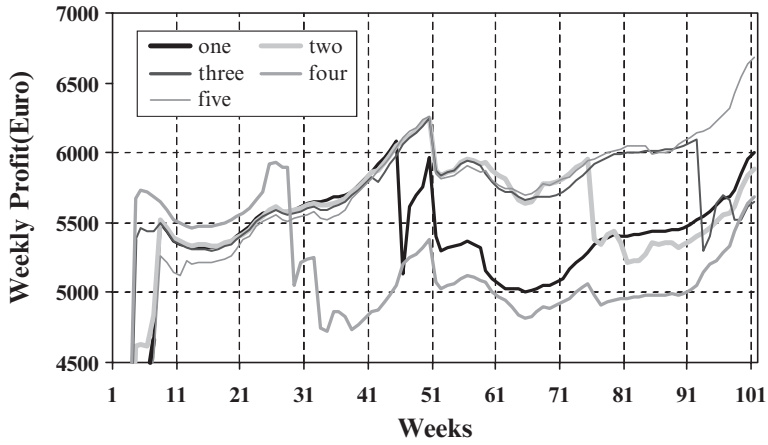

Fig. 6.18. Weekly profits made by a retailer equipped with PSO in 5 runs

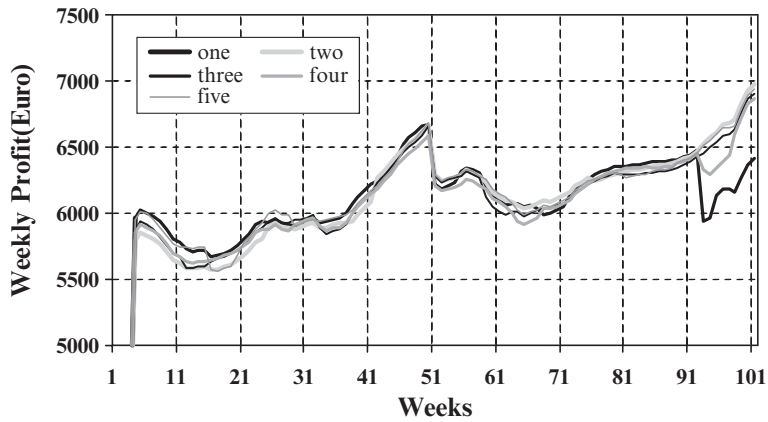

Fig. 6.19. Weekly profits made by a retailer equipped with EPSO in 5 runs

and MPGA were kept constant at 20 individuals while the population size of DCGA, SSGA, and PSO was increased to 100, 80, and 40 respectively.

Figure 6.22 shows the result as an average of five runs. Clearly, still EPSO is the winner and SSGA emerges now as the second best, while DCGA is confirmed as being the poor performer.

Figure 6.23 allows us to see that the best run of SSGA outperformed all others but that on average the profits made by a retailer with EPSO are stable on all runs and better than achieved with any other algorithm. Figure 6.24 clearly shows that now the computing effort of all five algorithms is relatively balanced.

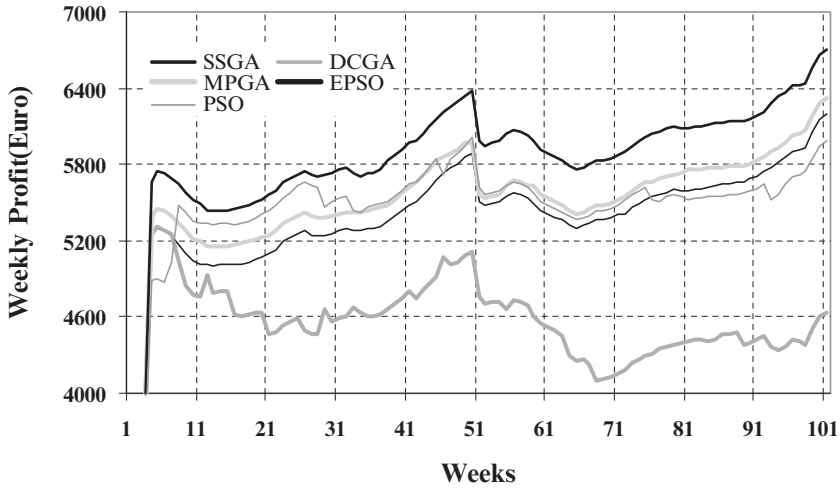

Fig. 6.20. Average of five runs for all algorithms

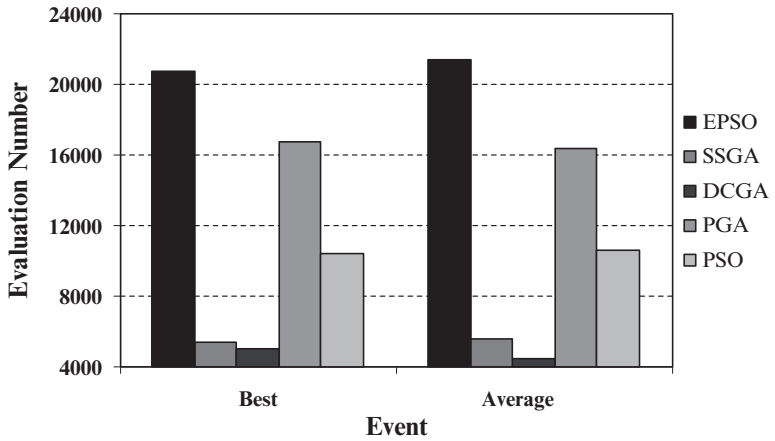

Fig. 6.21. Number of evaluations in the best run and on average of simulations done by the competing algorithms. Some algorithms experienced early termination because they could not achieve progress in a number of iterations

6.7 Conclusions

This chapter presents new ideas for improving the Evolutionary Particle Swarm Optimization algorithm and displays new evidence of the robustness of the method (in achieving consistently the same good results). The two innovations are: a random parameter affecting communication among particles, creating a topology that we call *stochastic star*, which proved valuable in cases where the algorithm needs to escape premature convergence; and a

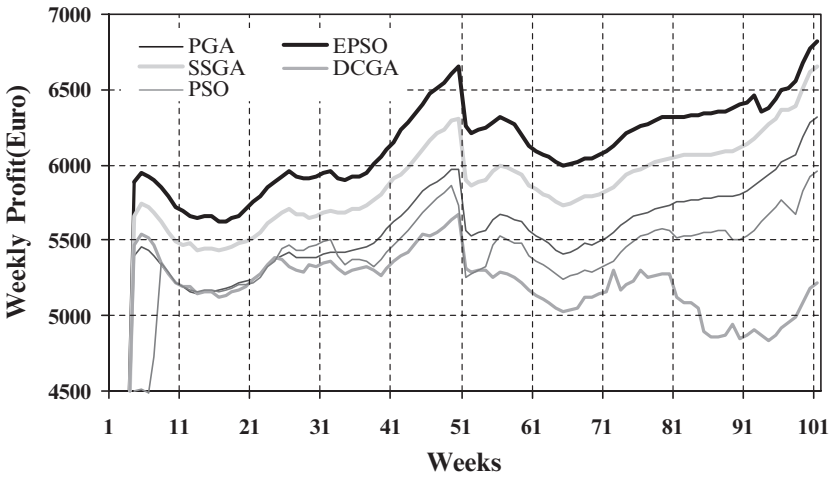

Fig. 6.22. Average of five runs with different population sizes (Exp. 2)

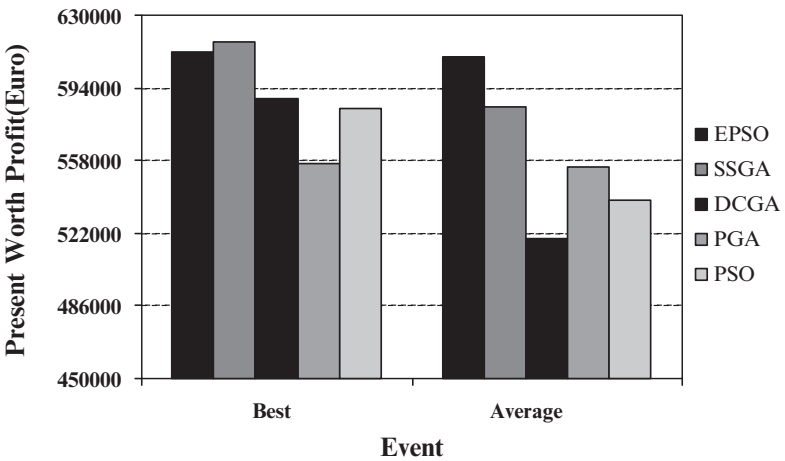

Fig. 6.23. Accumulated profits by a retailer equipped with each algorithm, for the best run and on the average of five runs of 2 year simulations (Exp. 2)

variant inspired in differential evolution that we call dEPSO, which displayed interesting behavior in the mixed integer problem of unit commitment which is a rather difficult practical problem in Power Systems.

The chapter provides also new evidence of the interest of the EPSO approach by reporting some results of a competition among distinct meta-heuristics in an extremely complex and dynamic environment: a market simulation platform composed of intelligent agents. The meta-heuristics were used to optimize the market strategy of retailer agents and EPSO emerged as the

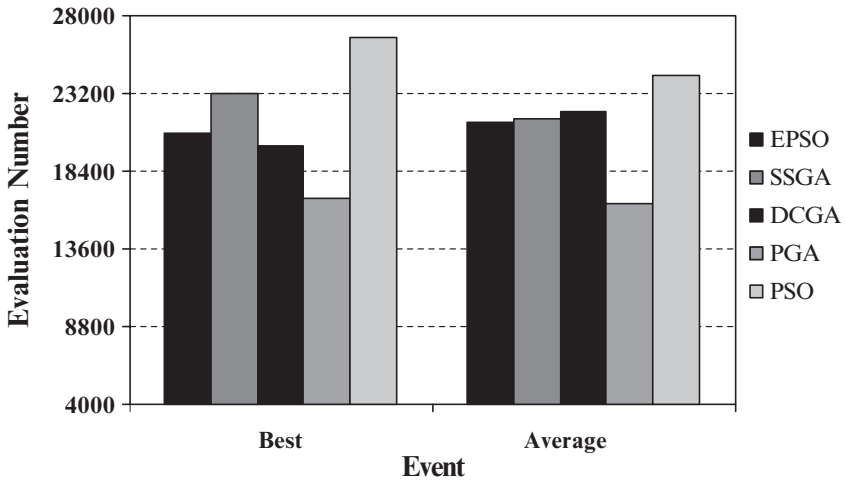

Fig. 6.24. Number of evaluations in the best run and on average of simulations done by the competing algorithms with different population sizes (Exp. 2)

algorithm that provided competitive advantage to the retailer using it to predict and select market decisions.

These results provide new incentive to further research the potential of the algorithm.

Acknowledgement

The authors wish to recognize the work of Nuno Fonseca, Naing Win Oo and Cristina Cerqueira, researchers at INESC Porto, who contributed at several stages to extensive testing of EPSO variants.

The research work has been partially funded by FCT – Foundation for Science and Technology of Portugal, under contract **POSC/EEA-ESE/60980/2004**.

References

1. V. Miranda and N. Fonseca, "EPSO – Best-of-Two-Worlds Meta-Heuristic Applied to Power System Problems", *Proceedings of WCCI/CEC – World Conference on Computational Intelligence, Conference on Evolutionary Computation*, Honolulu (Hawaii), USA, June 2002
2. V. Miranda and N. Fonseca, "Reactive Power Dispatch with EPSO - Evolutionary Particle Swarm Optimization", *Proceedings of PMAPS – International Conference on Probabilistic Methods Applied to Power Systems*, Naples, Italy, Sep 2002

3. M.S. Dias, N. Fonseca, H.M. Salgado and V. Miranda, "Optical Fibre Bragg Gratings Synthesis by Evolutionary Particle Swarm Optimization", *Proceedings of the Portuguese Physics Conference FISICA 2002*, Évora, Portugal, Sep 2002
4. V. Miranda and N. Fonseca, "EPSO - Evolutionary Particle Swarm Optimization, a New Algorithm with Applications in Power Systems ", Proceedings of IEEE/PES Transmission and Distribution Conference and Exhibition 2002: Asia Pacific, vol.2, pp.745–750, October, 2002
5. Mendonça, N. Fonseca, J.P. Lopes and V. Miranda, "Robust Tuning of Power System Stabilizers Using Evolutionary PSO", presented at ISAP 2003 – Intelligent Systems Applications to Power Systems Conference, Lemnos, Greece, September 2003
6. J.J. Xu and Z.H. Xin, "An Extended Particle Swarm Optimizer", Proceedings of the 19th IEEE International Parallel and Distributed Processing Symposium IPDPS'05 – workshop 6, Denver, Colorado, USA, April 2005
7. G.C. Chen and J.S. Yu, "Enhanced Particle Swarm Optimization and its Application in Soft Sensor" (in Chinese), Control And Decision, vol 20, no. 4, pp. 377–381, WangFa Data (ChinaInfo) and Chinese Electronic Periodical Services CEPS, April 2005
8. G. Yang and R. Zhang, "An Emotional Particle Swarm Algorithm", in Advances in Natural Computation: Proceedings of the First International Conference ICNC 2005, part III, page 553, Lecture Notes in Computer Science, Springer Berlin/Heidelberg, 2005
9. H.-P. Schwefel, "Adaptive Mechanismen in der biologischen Evolution und ihr Einfluß auf die Evolutionsgeschwindigkeit", Technical report, Technical University Berlin, 1974
10. D.B. Fogel, "Evolving Artificial Intelligence", Ph.D. Thesis, University of California, San Diego, 1992
11. H.-G. Beyer, "Toward a Theory of Evolution Strategies: Self-Adaptation", in *Evolutionary Computation*, vol. 3, no. 3, pp. 311–347, 1996
12. S. Meyer-Nieberg and H.-G. Beyer, "Self-adaptation in evolutionary algorithms" in F. Lobo, C. Lima, and Z. Michalewicz, editors, *Parameter Setting in Evolutionary Algorithm*s, Springer, series: Studies in Computational Intelligence , Vol. 54, 2007
13. J. Kennedy and R.C. Eberhart, "Particle Swarm Optimization", *Proceedings of the 1995 IEEE International Conference on Neural Networks*, pp. 1942–1948, Perth, Australia, 1995
14. M. Clerc, "The Swarm and the Queen: Towards a Deterministic and Adaptive Particle Swarm Optimization", *Proceedings of the 1999 Congress of Evolutionary Computation*, vol. 3, 1951–1957, Washington D.C., USA, IEEE Press, 1999
15. J. Kennedy, "The Particle Swarm: Social Adaptation of Knowledge", *Proceedings of the 1997 International Conference on Evolutionary Computation*, pp. 303–308, IEEE Press, 1997
16. Mendes, R., Kennedy, J., Neves, J., "The fully informed particle swarm: Simple, maybe better". IEEE Transactions on Evolutionary Computation 8, pp. 204–210, 2004
17. S. Mohais, R. Mendes, C. Ward and C. Postho, "Neighborhood re-structuring in particle swarm optimization", AI 2005 – 18th Australian joint conference on artificial intelligence, Sydney, Australia, December 5-9, 2005, in Lecture Notes in Computer Science (ISSN 0302-9743), Springer, 2005

18. R. Storn and K. Price, "Differential Evolution - A simple and efficient adaptive scheme for global optimization over continuous spaces", International Computer Science Institute – *Technical Report* TR-95-012, March 1995
19. M. Clerc et al., Standard PSO 2006, in Particle Swarm Central, http://www.particleswarm.info/, code retrieved Dec. 2006
20. S. Kitagawa and Y. Fukuyama, "Comparison of particle swarm optimizations for optimal operational planning of energy plants", Proceedings of the Swarm Intelligence Symposium 2005, pp. 155–161 June 2005
21. H. Mori and Y. Komatsu, "A Hybrid Method of Optimal Data Mining and Artificial Neural Network for Voltage Stability Assessment", Proceedings of IEEE St. Petersburg PowerTech, Russia, June 2005
22. N. W. Oo and V. Miranda, "Multi-energy Retail Market Simulation with Intelligent Agents", Proceedings of IEEE St. Petersburg Power Tech, St. Petersburg, Russia, June 2005
23. V. Miranda and N. W. Oo, "Evolutionary Algorithms and Evolutionary Particle Swarms (EPSO) in Modeling Evolving Energy Retailers", *Proceedings of PSCC - 15th Power Systems Computation Conference*, Liège, Belgium, Aug 2005
24. N. W. Oo and V. Miranda, "Evolving Agents in a Market Simulation Platform – A Test for Distinct Meta-Heuristics", Proceedings of the 13th International Conference on Intelligent Systems Application to Power Systems ISAP 2005, pp. 482–487, Arlington (VA), USA, Nov. 2005